

Internet de las Cosas y sus aplicaciones

Taller IoT: Nodos IoT simulados hablando con Internet

Simulación con ESP32 + Wokwi +
Cloudflare Tunnel+ REST SaaS Flask

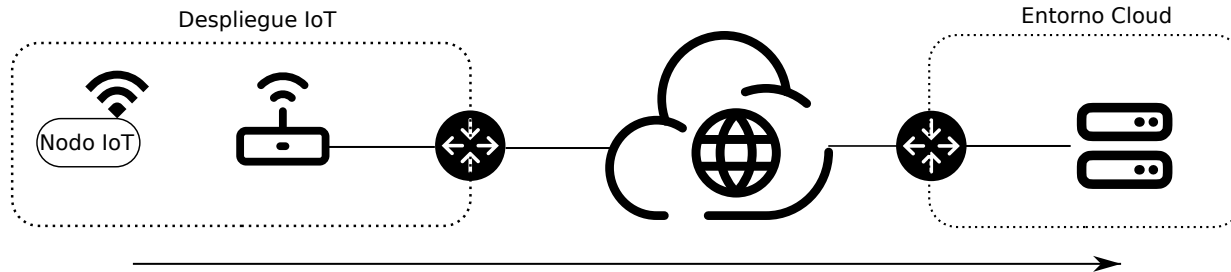
Felix.villanueva@uclm.es

Objetivo del Taller

- Simular un despliegue IoT real : wokwi
- Un dispositivo envía datos a un servidor SaaS en Internet con un API REST expuesta
- Comunicación cifrada HTTPS
- El servidor almacena vuestro ID
-

Qué vais a programar

- Cliente en ESP32 en wokwi
- Conexión WiFi
- Petición REST HTTPS a servidor FLASK
- Envío de parámetros en URL

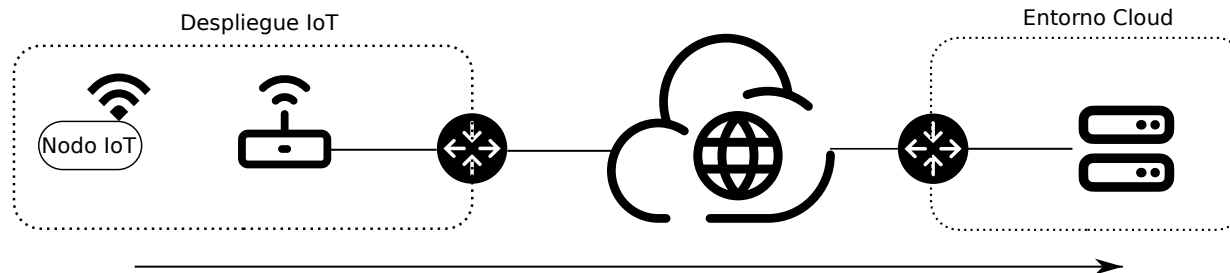


Qué es una API REST

- Un servidor que recibe peticiones HTTP
- El cliente solicita una URL
- El servidor responde
- El ESP32 actúa como un navegador muy pequeño que le da los datos de los sensores (ID) en la invocación.
- Todos los firewalls dejan pasar el protocolo HTTP ;-)

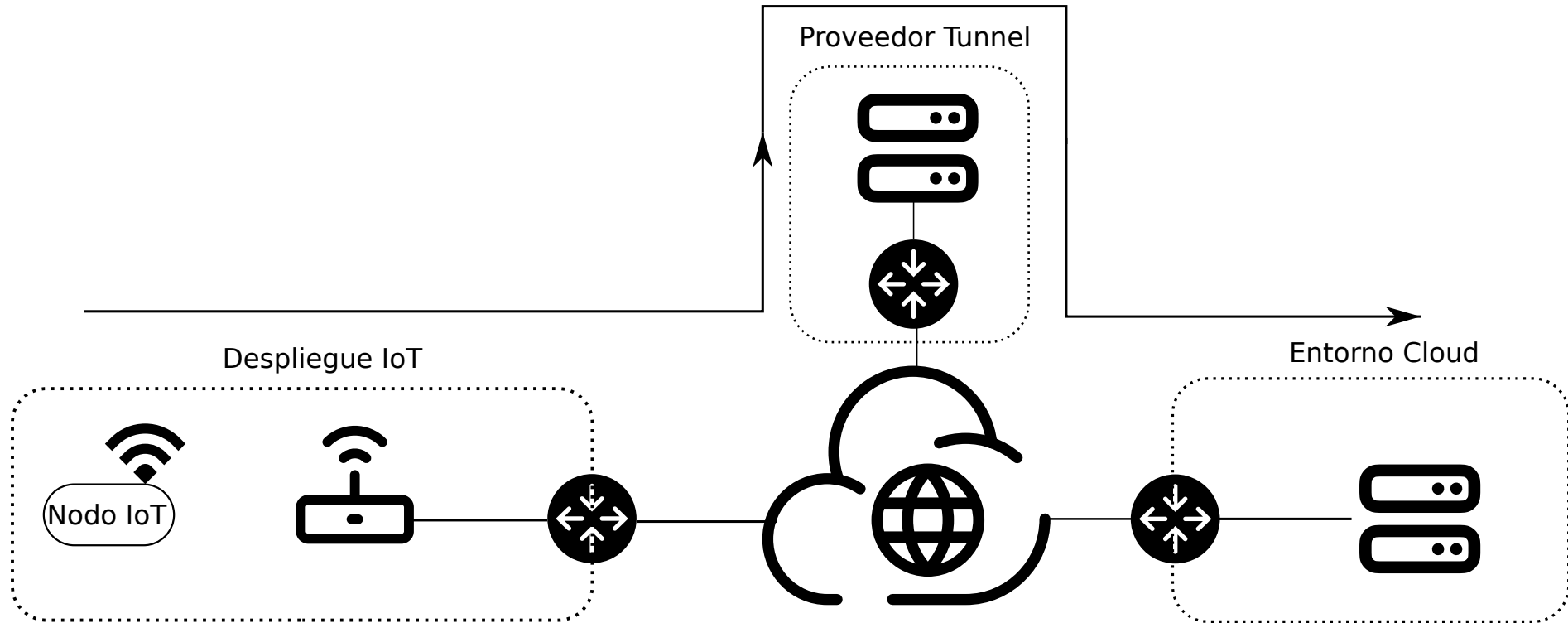
Problema real

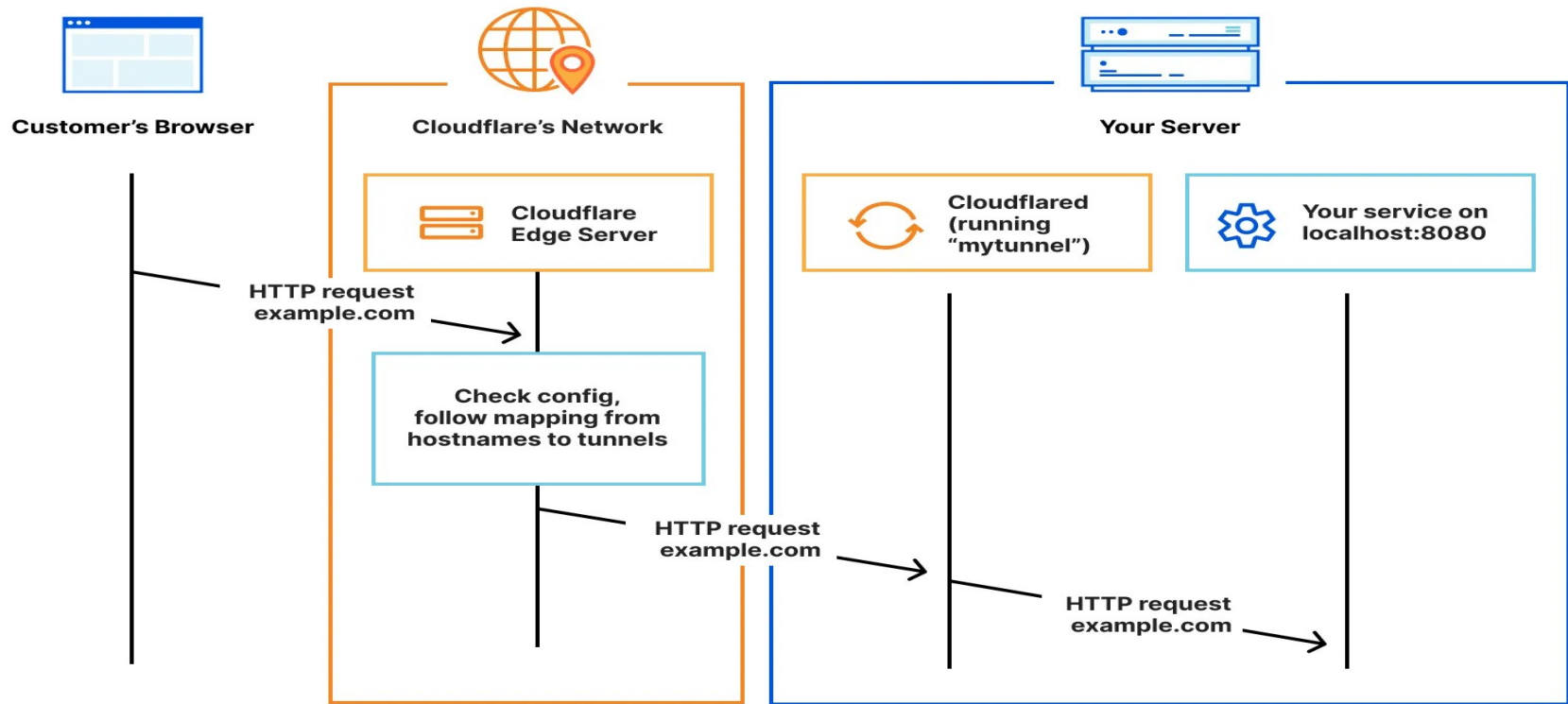
- El servidor está en ordenador con IP privada
- No tengo IP pública
- El router bloquea conexiones entrantes
- Los dispositivos no pueden acceder directamente



Solución: túnel inverso

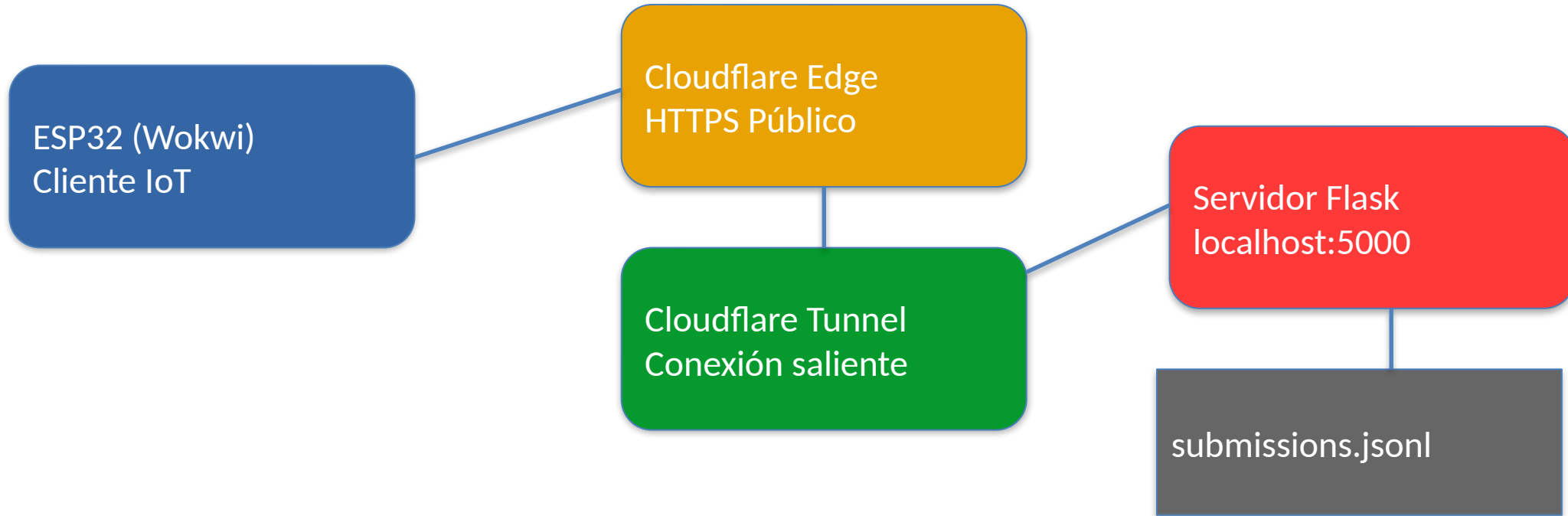
- El ordenador que aloja el servidor se conecta hacia fuera
- Usaremos el proveedor Cloudflare
- Cloudflare publica una URL
- Los alumnos se conectan a Cloudflare
- Cloudflare reenvía la petición a mi ordenador





<https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>

Arquitectura del sistema



Establecer el tunel inverso

- Cuenta en cloudflare, instalar la herramienta cloudflared ([ubuntu](#)), hacer :
- `$cloudflared tunnel login`
 - ~ Solo se hace una vez
 - ~ Genera un cert.pem para tu máquina
- Arrancas el servidor (e.j. `http://localhost:5000`) y vinculas el tunel a la dirección local/puerto (Transparencia siguiente) con la herramienta cloudflared

```
Felix@homer:~/tools/cloudflare$ cloudflared tunnel --url http://localhost:5000
2026-02-17T10:16:28Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way to experiment and try it out. However, be aware that these accounts-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-02-17T10:16:28Z INF Requesting new quick Tunnel on trycloudflare.com...
2026-02-17T10:16:31Z INF +-----+
2026-02-17T10:16:31Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): |
2026-02-17T10:16:31Z INF | https://gazette-deaths-publicity-colorado.trycloudflare.com |
2026-02-17T10:16:31Z INF +-----+
2026-02-17T10:16:31Z INF Version 2026.2.0 (Checksum 176746db3be7dc7bd48f3dd287c8930a4645ebb6e6700f883fddda5a4c307c16)
2026-02-17T10:16:31Z INF GOOS: linux, GOVersion: go1.24.13, GoArch: amd64
2026-02-17T10:16:31Z INF Settings: map[cred-file:/home/felix/.cloudflared/1bd2bd91-00be-4eb0-9758-ed5ffc49443c.json credentials-file:/home/felix/.cloudflared/1bd2bd91-00be-4eb0-9758-ed5ffc49443c.json ha-connections:1 protocol:quic url:http://localhost:5000]
2026-02-17T10:16:31Z INF cloudflared will not automatically update if installed by a package manager.
2026-02-17T10:16:31Z INF Generated Connector ID: 31cf2c7a-5c3b-42d5-ab40-b92aaf4ba72b
2026-02-17T10:16:31Z INF Initial protocol quic
2026-02-17T10:16:31Z INF ICMP proxy will use 172.24.27.38 as source for IPv4
2026-02-17T10:16:31Z INF ICMP proxy will use fe80::15e5:42ec:7193:43bb in zone enp4s0 as source for IPv6
2026-02-17T10:16:31Z INF ICMP proxy will use 172.24.27.38 as source for IPv4
2026-02-17T10:16:31Z INF ICMP proxy will use fe80::15e5:42ec:7193:43bb in zone enp4s0 as source for IPv6
2026-02-17T10:16:31Z INF Starting metrics server on 127.0.0.1:20242/metrics
2026-02-17T10:16:31Z INF Tunnel connection curve preferences: [X25519MLKEM768 CurveP256] connIndex=0 event=0 ip=198.41.192.167
2026/02/17 11:16:31 failed to sufficiently increase receive buffer size (was: 208 kiB, wanted: 7168 kiB, got: 416 kiB). See https://github.com/quic-go/quic-go/wiki/UDP-Buffer-Sizes for details.
2026-02-17T10:16:32Z INF Registered tunnel connection connIndex=0 connection=bb48ab9d-bcbd-4019-9efb-353c3e934520 event=0 ip=198.41.192.167 location=mad06 protocol=quic
```

□

Qué datos enviáis

- DNI y TOKEN de seguridad en formato JSON
- `snprintf(body, sizeof(body),
 "{\"dni\": \"%s\", \"token\": \"%s\"}\", DNI, TOKEN);`
- Método POST

```
felix@homer:~$ curl -X POST http://127.0.0.1:5000/submit -H "Content-Type: application/json" -d '{"token":"UCLM-IOT-5555","dni":"12345678A"}'  
{  
  "dni": "12345678A",  
  "ok": true,  
  "saved": true  
}  
felix@homer:~$
```

Qué ocurre en el servidor

- Flask recibe la petición POST
- Extrae el parámetro dni
- Extrae el TOKEN de Seguridad y comprueba que es válido
- Guarda una línea en un archivo submissions.jsonl
- Registro permanente

Archivo submissions.jsonl

- Cada petición = una línea
- Formato JSON
- Permite verificar entregas
- Se puede analizar después

Qué conceptos estáis aprendiendo

- IoT real
- HTTPS
- Arquitectura cliente-servidor
- Computación en la nube

Entrega (9 marzo 2026)

- En el submission.json debe aparecer vuestro dni
- En la tarea de campus virtual, subir un zip con el código del simulador wokwi y el esquema.
- Si cambia el código y/o la URL se avisará por campus virtual.